

Rapport Final Global : Applications Distribuées & Big Data

Nom et Prénom : Zakaria Ech-chafi

Module : Applications Distribuées & Big Data

Année Universitaire : 2025-2026

1. Introduction

Le module « Applications Distribuées & Big Data » a permis d'aborder les défis liés à la conception de systèmes capables de traiter des volumes massifs de données en temps réel ou en différé. L'objectif était de maîtriser la chaîne de valeur de la donnée, depuis son ingestion jusqu'à son exploitation, tout en garantissant robustesse, scalabilité et traçabilité. Ce rapport synthétise les travaux réalisés lors des différentes séances.

2. Synthèse des travaux (TPs)

TP 1 & 2 : Modélisation et Qualité des données (Data Engineering)

Objectifs : Maîtriser la modélisation objet en Python pour structurer les échanges de données.

Démarche : Utilisation des `dataclasses` (notamment `frozen=True` pour l'immuabilité) et des *type hints* pour assurer la robustesse du code. Mise en place de protocoles de validation pour nettoyer les flux entrants (ex: `errors='coerce'`) afin de garantir la qualité des données avant leur ingestion dans le système.

Résultats : Création d'une couche d'ingestion capable de rejeter les données malformées, assurant ainsi la fiabilité des futurs traitements Big Data.

TP 3 & 4 : Communication et RPC (Applications Distribuées)

Objectifs : Établir une communication inter-processus fiable dans un environnement distribué.

Démarche : Implémentation de sockets TCP avec des contrats de messages explicites (format JSON). Mise en place d'un modèle RPC (Remote Procedure Call) permettant à un client d'appeler des fonctions distantes de manière transparente.

Résultats : Développement d'un système capable de traiter des requêtes à distance, incluant une gestion fine de la latence et de l'identification unique des requêtes via `request_id` (corrélation).

TP 5 & 6 : Architecture Événementielle et Streaming

Objectifs : Passer d'un traitement par lot (batch) à un traitement en flux continu (stream processing).

Démarche : Mise en œuvre du pattern Producteur-Consommateur via des files de messages (Pub/Sub). Utilisation d'agrégations en temps réel (sommes, comptages) sur des fenêtres temporelles pour analyser le débit (*throughput*) et la latence du système.

Résultats : Un pipeline capable de traiter des événements au fil de l'eau, avec une gestion du *backpressure* pour ralentir la source lorsque le consommateur est saturé.

TP 7 & 8 : Distribution et Tolérance aux pannes

Objectifs : Assurer la scalabilité et la persistance des données dans un système distribué.

Démarche : Implémentation du partitionnement des données par clé pour paralléliser la lecture. Gestion des *offsets* et du *commit* pour permettre la reprise (*recovery*) après une panne, garantissant ainsi une sémantique de traitement *at-least-once*.

Résultats : Un système résilient capable de redémarrer après un arrêt tout en retrouvant son état exact grâce aux sauvegardes périodiques (*checkpoints*).

TP 9 : Architecture Big Data

Objectifs : Concevoir une architecture globale cohérente.

Démarche : Synthèse des acquis pour structurer un *Data Lake* organisé, évitant le problème du "Data Swamp" (marécage de données) par une gouvernance stricte des zones de stockage.

Résultats : Mise en place d'une traçabilité (*lineage*) des données, permettant de remonter de la source à la transformation finale pour chaque élément traité.

3. Analyse personnelle

Le parcours au sein de ce module a été particulièrement exigeant, marqué par une courbe d'apprentissage abrupte. La densité du programme, alliant concepts théoriques abstraits et implémentations techniques, a représenté un défi quotidien.

Parmi les difficultés rencontrées, j'ai dû appréhender un grand nombre de notions nouvelles dont la complexité était parfois intimidante. La transition vers des architectures distribuées demande un changement de paradigme complet par rapport à la programmation classique : là où un programme monolithique est prévisible, un système distribué introduit des variables imprévisibles, comme la latence réseau ou les pannes partielles, qu'il faut apprendre à gérer dès la conception.

La compréhension fine de certains cours a été ardue en raison de la nature technique des sujets abordés, tels que la gestion précise des *offsets* dans un système de streaming ou la mise en œuvre de protocoles RPC asynchrones. Toutefois, c'est précisément cette complexité qui a été la plus formatrice. En cherchant des solutions aux erreurs réseau, en déboguant des comportements asynchrones et en structurant mes premières pipelines de données, j'ai transformé ces obstacles en acquis techniques concrets.

Ce module ne m'a pas seulement apporté des connaissances théoriques ; il m'a surtout appris la persévérance face à des concepts complexes. La capacité à décomposer un problème distribué en sous-systèmes isolés et testables est, selon moi, l'apprentissage le plus précieux que je retiens de ce semestre.

4. Conclusion générale

Ce module m'a permis de comprendre qu'une plateforme Big Data n'est pas un simple assemblage d'outils, mais une architecture raisonnée où chaque brique (contrat, réseau, partitionnement) répond à une contrainte spécifique. Le passage d'une vision "script" à une vision "système distribué" constitue l'apport principal de ce semestre.